

Fasal Guard

Project Team

Daniyal Khawar	22I-1066
Muhammad Iqrash Qureshi	22I-1174
Hammad Shabbir	22I-1140

Session 2022-2026

Supervised by

Dr. Labiba Fahad



Department of Computer Science

**National University of Computer and Emerging Sciences
Islamabad, Pakistan**

June, 2026

Contents

1	Introduction	1
1.1	Project Overview	1
1.2	Problem Statement	1
1.3	Scope	2
1.4	Modules	3
1.4.1	Module 1: Data Collection	3
1.4.2	Module 2: Data Cleaning and Preprocessing	4
1.4.3	Module 3: Yield Forecasting (Iteration 1)	4
1.4.4	Module 4: Soil Integration & Satellite Images Collection (Iteration 2)	5
1.4.5	Module 5: Satellite Images & Hybrid Model Development (Iteration 3)	5
1.4.6	Module 6: Stress Detection and Climate-Risk Forecasting	5
1.4.7	Module 7: Alerts, Dashboard and Deployment (Iteration 4)	6
1.5	User Classes and Characteristics	6
2	Project Requirements	9
2.1	Use-case	9
2.1.1	Use Case 1: Signup / Create an Account	11
2.1.2	Use Case 2: Forgot Password / Reset Password	11
2.1.3	Use Case 3: Give Feedback/ Ask Query	13
2.1.4	Use Case 4: Advising Actions After Soil Health Analysis	13
2.1.5	Use Case 5: Weather Prediction	15
2.1.6	Use Case 6: Alerts/ Notifications	17
2.1.7	Use Case 7: View Crop Field Forecasts / Crop Stress Heatmaps	19
2.1.8	Use Case 8: Predict Disaster and Its Precautionary Measures	21
2.2	Functional Requirements	23
2.2.1	Module 1: Data Collection	23
2.2.2	Module 2: Data Cleaning and Preprocessing	23
2.2.3	Module 3: Yield Forecasting (Iteration 1)	24
2.2.4	Module 4: Soil Integration and Satellite Images Collection (Iteration 2)	24

2.2.5	Module 5: Satellite Images and Hybrid Model Development (Iteration 3)	24
2.2.6	Module 6: Stress Detection and Climate-Risk Forecasting	25
2.2.7	Module 7: Alerts, Dashboard, and Deployment (Iteration 4)	25
2.3	Non-Functional Requirements	26
2.3.1	Security	26
2.3.2	Usability	26
2.3.3	Responsiveness	26
2.3.4	Availability	26
2.3.5	Reliability	27
2.3.6	Performance	27
2.3.7	Model Accuracy	27
2.3.8	Maintainability	27
2.3.9	Scalability	27
2.4	Domain Model	28
3	System Overview	31
3.1	Architectural Design	31
3.2	Design Models	33
3.2.1	Activity Diagram	33
3.2.2	Class Diagram	35
3.2.3	Class-Level Sequence Diagram	37
3.3	Data Design	43
3.3.1	Database Architecture	43
3.3.2	Entity-Relationship Model	44
3.3.3	Data Processing Pipeline	44
3.3.4	Data Storage and Access	46
3.3.5	Data Security and Privacy	46
3.3.6	Data Integration and APIs	47
3.3.7	Scalability Considerations	48

List of Figures

2.1	Use Case Diagram for FasalGuard	10
2.2	Domain Model of FasalGuard	29
3.1	Architectural Diagram	32
3.2	Activity Diagram of FasalGuard	34
3.3	Class Diagram of FasalGuard	36
3.4	Class-Level Sequence Diagram of SignUp/CreateanAccount	38
3.5	Class-Level Sequence Diagram of Weather Prediction	39
3.6	Class-Level Sequence Diagram of Forgot Password	40
3.7	Class-Level Sequence Diagram of Feedback/Ask Query	41
3.8	Class-Level Sequence Diagram of LSTM model	42

List of Tables

1.1	User Classes and Characteristics	7
2.1	Use Case 1: Signup / Create an Account	11
2.2	Use Case 2: Forgot Password / Reset Password	12
2.3	Use Case 3: Feedback/Ask Query	13
2.4	Use Case 4: Advising Actions After Soil Health Analysis	14
2.5	Use Case 5: Weather Prediction	16
2.6	Use Case 6: Alerts and Notifications	18
2.7	Use Case 7: View Crop Field Forecasts / Crop Stress Heatmaps	20
2.8	Use Case 8: Predict Disaster and Its Precautionary Measures	22

Chapter 1

Introduction

FasalGuard is an AI-driven precision agriculture system that aims to revolutionize the agricultural sector through data-driven insights. It integrates climate, soil, and satellite data to monitor crop health, forecast yields, and provide early warnings about potential risks such as droughts or pest outbreaks. The system empowers farmers and agricultural authorities to make informed, timely, and sustainable decisions for improved productivity and resource optimization.

1.1 Project Overview

FasalGuard is designed to help farmers, agricultural organizations, and policymakers make informed, data-backed farming decisions. The system integrates multiple heterogeneous data sources such as climate data (NASA POWER), annual production data (Pakistan Bureau of Statistics), soil datasets (ISRIC SoilGrids) and satellite imagery (Sentinel-2) to monitor crop health, forecast yields, and detect potential environmental or stress-related risks. Through advanced machine learning and deep learning models, FasalGuard analyzes patterns in soil fertility, weather trends, and vegetation indices to generate actionable insights for better crop management. The platform provides an intuitive web dashboard and SMS alert system to deliver timely recommendations and early warnings.

1.2 Problem Statement

Agriculture is the backbone of Pakistan's economy, contributing significantly to GDP and employment, but it remains one of the most vulnerable sectors to climate change, resource mismanagement, and unpredictable weather patterns. Farmers often rely on traditional knowledge and manual observation to make critical decisions about irrigation,

fertilization, and pest control. This results in delayed actions, inefficient resource use, and significant yield losses.

Currently, there is no unified digital platform in Pakistan that provides integrated, real-time, and localized agricultural intelligence by combining climate data, soil parameters, and satellite imagery. Existing systems tend to focus on isolated factors such as weather forecasts or soil reports, which fail to capture the dynamic and interconnected nature of crop growth. Consequently, farmers lack the situational awareness and predictive insights needed to respond proactively to changing environmental conditions.

FasalGuard is being developed to address these gaps by introducing an AI-driven precision agriculture solution capable of monitoring crop health, predicting yields, and identifying potential risks such as drought, nutrient deficiency, or pest infestation. By integrating multiple data sources and applying advanced machine learning models, the system generates field-level forecasts and stress maps that can guide farmers toward optimal irrigation, fertilizer, and harvesting decisions.

The platform further enhances accessibility through a user friendly web application, ensuring that insights reach farmers regardless of technical literacy. This system directly tackles the issues of low productivity, water scarcity, and climate vulnerability, offering actionable and data-backed recommendations to improve efficiency and sustainability.

Ultimately, FasalGuard seeks to bridge the technological gap between modern agricultural research and on-ground farming practices, empowering farmers to make smarter, faster, and more sustainable decisions that lead to higher yields, lower costs, and greater resilience against climate-induced risks.

1.3 Scope

The scope of FasalGuard encompasses the design and development of an AI-based agriculture system that integrates multi-source agricultural data to assist farmers and agricultural authorities in making informed decisions. The project aims to collect and harmonize data from diverse sources, including satellite imagery (Sentinel-2), climate data (NASA POWER), and soil datasets (ISRIC SoilGrids and pilot field samples). The collected data will be cleaned, preprocessed, and used to train deep learning models for accurate yield forecasting and stress detection at the field level. Data will be collected for various districts of Punjab. Currently, the system will make predictions for major crops that are grown in the region i.e. corn, maize, sugarcane, wheat and rice. The system will provide real-time insights and recommendations related to irrigation needs, nutrient deficiencies, and potential climate risks such as drought or flood conditions.

The core functionalities of the system are:

- Acquire diverse agricultural datasets, including satellite imagery, soil health metrics (pH, nitrogen), and climate trend.
- Preprocess heterogeneous data sources for consistency, noise removal, and model readiness.
- Train hybrid AI models (LSTM for climate, Random Forest for soil, CNN for satellite imagery) to forecast crop yields and detect stress.
- Integrate climate-risk forecasts (e.g, droughts, floods) with soil and crop data for resource-efficient recommendations.
- Generate intuitive visualizations such as hyperlocal stress heatmaps and yield risk assessments.
- Deliver real-time alerts via SMS for critical events and provide a user-friendly dashboard for holistic farm monitoring.
- Deploy and validate the system in pilot fields across Punjab, focusing on high impact crops like wheat, rice, cotton, and sugarcane.

FasalGuard will focus on predictive analytics and decision support for farm management, exclude direct hardware interventions (e.g., automated irrigation systems), and prioritize scalable digital outputs accessible to farmers through dashboards and SMS alerts.

1.4 Modules

The proposed project FasalGuard is divided into a series of interconnected modules. Each module addresses a specific stage in the precision agriculture pipeline, beginning with data acquisition and preprocessing, and progressing toward hybrid AI modeling, visualization, and deployment. The system begins with data collection, processing the data and training the model alongside development of various website features like signing up, OTP generation for verification, contact us, dashboard creation, weather prediction etc. All modules together constitute the Client Web Application and its supporting backend components.

1.4.1 Module 1: Data Collection

This module focuses on acquiring and organizing diverse agricultural datasets required for analysis and model training.

1. Collect 10+ years of historical climate data for Punjab from NASA POWER and yearly production data from PBC.
2. Acquire crop yield statistics for major crops such as wheat, rice, maize, cotton, and sugarcane.
3. Gather soil health parameters (e.g., pH, nitrogen, moisture, and organic matter) from pilot fields.
4. Retrieve Sentinel-2 satellite imagery for field-level vegetation monitoring and crop classification.
5. Store all collected data in a structured database and file repository for easy access and integration.

1.4.2 Module 2: Data Cleaning and Preprocessing

This module ensures the quality, consistency, and usability of heterogeneous datasets obtained from multiple sources.

1. Handle missing values, noise, and inconsistencies in the data.
2. Normalize and scale soil metrics for comparability across different geographic regions.
3. Preprocess Sentinel-2 imagery through cloud masking, band selection, and spatial resampling.
4. Prepare cleaned datasets for model training and storage.

1.4.3 Module 3: Yield Forecasting (Iteration 1)

This module develops the baseline predictive model to forecast crop yields using historical datasets.

1. Train the time series data on different models and choose the one with the best accuracy.
2. Evaluate the model using metrics such as RMSE, MAE, and R^2 to establish benchmark performance.
3. Generate initial yield forecasts with historical trend visualizations and accuracy metrics.
4. Display model outputs and insights on the website's dashboard.

1.4.4 Module 4: Soil Integration & Satellite Images Collection (Iteration 2)

This module enhances predictive accuracy by integrating soil health and satellite imagery data into the modeling pipeline.

1. Train soil health features data to assess fertility and productivity levels on various deep learning models.
2. Evaluate the model using metrics such as RMSE, MAE, and R^2 to establish benchmark performance.
3. Combine the soil model prediction with yield forecasting and climate change predictions.
4. Generate real time images in order to train CNN-based model in next phase.

1.4.5 Module 5: Satellite Images & Hybrid Model Development (Iteration 3)

Development of CNN based model for real time analysis and integration of all predictive models into a single, optimized hybrid AI pipeline.

1. Implement CNN-based analysis of Sentinel-2 imagery to capture vegetation and stress patterns.
2. Combine LSTM (climate), Random Forest (soil), and CNN (satellite) models into a unified hybrid architecture.
3. Develop real-time prediction workflows for yield and crop stress analysis.
4. Implement rule-based alert logic for early detection of droughts, nutrient deficiencies, or pest risks.
5. Generate interactive heatmaps and risk warnings on the dashboard.
6. Optimize model performance for scalability and near-real-time analytics.

1.4.6 Module 6: Stress Detection and Climate-Risk Forecasting

This module focuses on improving climate resilience by modeling environmental risks and crop stress conditions.

1. Integrate real-time weather feeds with historical climate trends to detect anomalies.
2. Identify early indicators of drought, flood, or pest outbreaks using predictive modeling.
3. Generate hyperlocal stress heatmaps and visual risk assessments.
4. Provide actionable recommendations for irrigation scheduling, nutrient management, and pest control.
5. Support temporal forecasting to anticipate future climate-risk events at the field level.

1.4.7 Module 7: Alerts, Dashboard and Deployment (Iteration 4)

This module ensures real-world usability, farmer empowerment, and scalability.

1. Design a responsive dashboard for holistic farm monitoring.
2. Deploy the web app with actionable suggestions and user data integration.
3. Validate deployment on pilot farms and prepare for regional scaling.

1.5 User Classes and Characteristics

The FasalGuard system will be used by several classes of users, each playing a specific role in the collection, management, and utilization of agricultural intelligence. The following table summarizes the primary user classes, their characteristics, and their interactions with the system.

Table 1.1: User Classes and Characteristics

User Class	Description
Farmer (End User)	The primary beneficiary of FasalGuard. Farmers use the Client Web Application or SMS service to monitor crop health, view yield forecasts, and receive alerts about irrigation, pest risks, and nutrient deficiencies. They generally have limited technical literacy and prefer simple, localized information in their native language. Accessibility and timely SMS notifications are essential for this user group.
Agricultural Officer / Extension Worker	Agricultural officers and extension workers use the platform to assist groups of farmers by analyzing field data, interpreting heatmaps, and recommending interventions. They act as intermediaries between the system and the farmers, promoting adoption and providing on-ground validation. These users are moderately technical and require access to field-level dashboards.
Admin User	Administrators manage datasets, user accounts, model retraining, and alert configurations. They monitor the backend system for performance, reliability, and data consistency. This class includes project supervisors, software maintainers, and technical staff responsible for ensuring the system's continuous operation. Admin users interact with the Admin Web Application.
Government Agencies / Policy Makers	This user class accesses aggregated data, regional forecasts, and summary dashboards to support decision-making for disaster preparedness, crop insurance, and water management. They require access to analytical and statistical reports rather than individual farm-level data. Security and data privacy are high priorities for this class.
Researchers / Academic Institutions	Researchers use FasalGuard's datasets and model outputs for analysis, validation, and further development of predictive models. They focus on performance metrics, algorithmic accuracy, and data quality. This group requires technical access to historical data and modeling modules for experimentation.
Agri-tech Companies / Investors	These users are potential commercial partners interested in scaling the platform or integrating it into existing digital agriculture systems. They require system demonstrations, access to performance summaries, and deployment scalability details to evaluate investment or partnership opportunities.
Satellite Data Providers	External service providers that supply imagery and geospatial datasets to FasalGuard. They ensure continuous data availability and quality, and may require limited access to system APIs for data integration and updates.

Chapter 2

Project Requirements

This chapter describes the functional and non-functional requirements of the project.

2.1 Use-case

Since FasalGuard is an interactive, web-based application with multiple user roles (Farmers and Admins), the **Use Case** technique is used to describe the main interactions between the system and its users. This section presents both the use case diagram and tabular representations of major user interactions implemented or planned in the first development iteration.

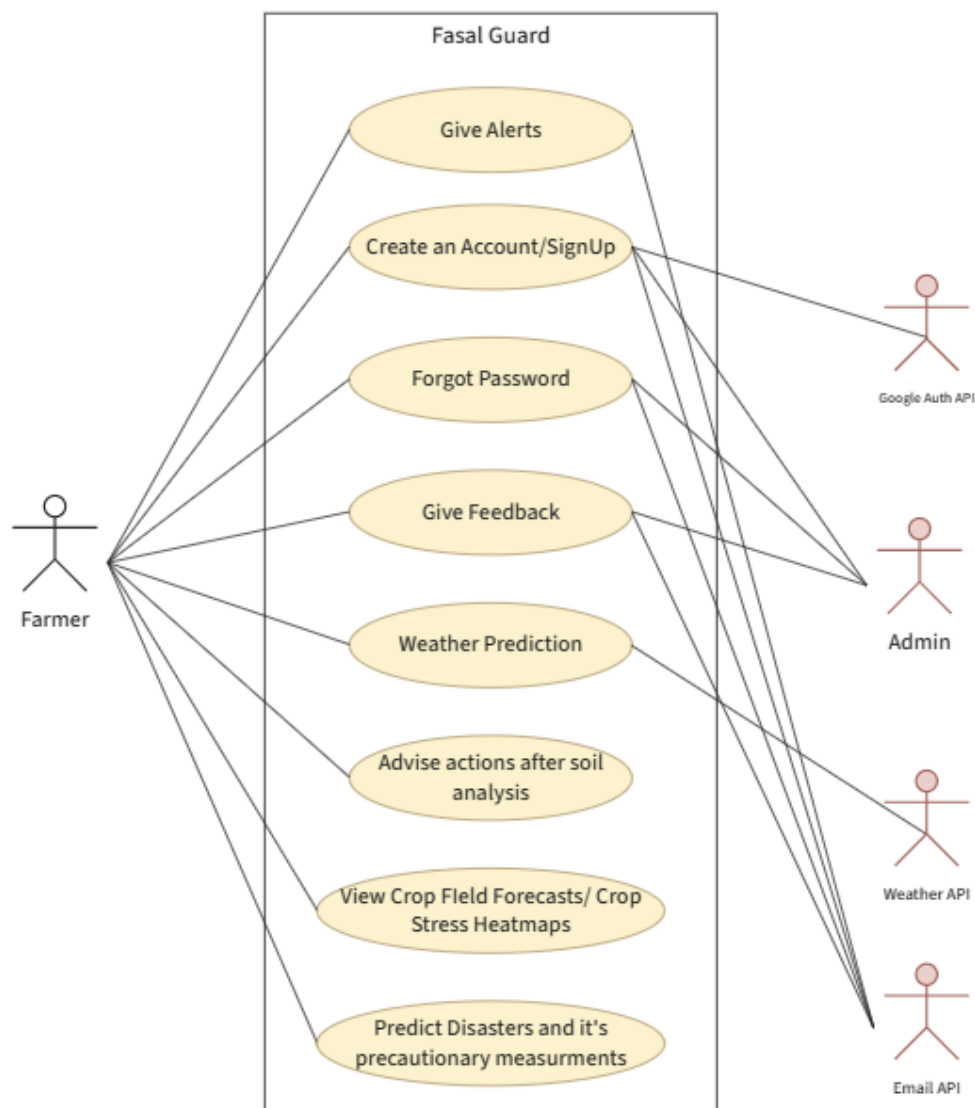


Figure 2.1: Use Case Diagram for FasalGuard

2.1.1 Use Case 1: Signup / Create an Account

This use case allows both Farmers and Admins to create an account in the FasalGuard system, either by filling out a registration form or by using Google authentication.

Table 2.1: Use Case 1: Signup / Create an Account

Use Case Name	Signup / Create an Account
Actors	Farmer (End User), Admin (System User)
Description	Allows a new user, either a Farmer or an Admin, to create an account on the FasalGuard platform. The user provides their Full Name, Email, and Password to register. An OTP (One-Time Password) is sent to the provided email for verification. Upon successful OTP confirmation, the account is created. Alternatively, the user may sign up directly using their Google account credentials.
Preconditions	1. User is not already registered. 2. User must have a valid email address.
Postconditions	1. A verified user account is created and stored in the database. 2. User is redirected to the login page or auto-logged in (Google signup).
Main Success Scenario	1. User opens the <i>Signup</i> page. 2. Enters Full Name, Email, and Password. 3. System sends an OTP to the provided email. 4. User enters OTP on the verification screen. 5. System verifies OTP and creates the account. 6. Confirmation message displayed: "Account successfully created."
Alternate Flow	1. Email already exists → Prompt to log in. 2. OTP invalid or expired → System allows resend. 3. "Sign up with Google" selected → Google API authenticates user; OTP skipped.
Extensions	1. Network or email delivery issue prevents OTP. 2. Database error during account creation.
Priority	High

2.1.2 Use Case 2: Forgot Password / Reset Password

This use case allows both Farmers and Admins to reset their password securely through email verification using a One-Time Password (OTP). The process ensures that only verified users can regain access to their accounts.

Table 2.2: Use Case 2: Forgot Password / Reset Password

Use Case Name	Forgot Password / Reset Password
Actors	Farmer (End User), Admin (System User)
Description	This use case enables a registered user who has forgotten their password to reset it securely. The user enters their registered email address, and the system sends a One-Time Password (OTP) to that email. Once the OTP is verified, the user can set a new password to regain access to their account.
Preconditions	1. The user must already be registered in the system. 2. The user must have access to their registered email address.
Postconditions	1. The user's password is successfully reset. 2. The user can log in using the new password.
Main Success Scenario	1. User clicks on the "<i>Forgot Password?</i>" link on the login page. 2. User enters their registered email address. 3. System validates the email and sends an OTP to it. 4. User enters the OTP received in their email. 5. System verifies the OTP. 6. User is prompted to create and confirm a new password. 7. System updates the password in the database and displays a success message.
Alternate Flow	1. Email not registered → System displays "Email not found." 2. OTP invalid or expired → System allows resend and re-verification. 3. User enters mismatched passwords → System prompts to re-enter password.
Extensions	1. Email delivery failure due to server issues. 2. Network connectivity problems during password reset process.
Priority	High

2.1.3 Use Case 3: Give Feedback/ Ask Query

This use case allows a logged-in Farmer to contact the system administrator or support team directly through a built-in “Contact Us” feature available on the navigation bar of the home page. The farmer can enter their name, email address, and message, which will be sent to the admin’s email for review and response.

Table 2.3: Use Case 3: Feedback/Ask Query

Use Case Name	Contact Us
Actors	Farmer (End User)
Description	This use case enables a logged-in Farmer to send feedback, queries, or issue reports directly to the Admin. The Farmer accesses the “Contact Us” form from the navigation bar, fills in their name, email, and message, and submits it. The system then sends the message to the Admin’s official email address.
Preconditions	1. The Farmer must be logged into the system. 2. Internet connectivity must be active.
Postconditions	1. The message is successfully delivered to the Admin’s email.
Main Success Scenario	1. Farmer clicks on the “ <i>Contact Us</i> ” option in the navigation bar. 2. System displays the contact form fields: Name, Email, and Message. 3. Farmer enters the required details and presses <i>Send Message</i> . 4. System validates all input fields (non-empty and valid email format). 5. System sends the message to the Admin’s registered email address.
Alternate Flow	3. One or more fields left empty → System prompts user to fill all required fields. 3. Invalid email format → System displays an “Invalid email address” error.
Extensions	1. Internet connectivity loss before message submission.
Priority	Medium

2.1.4 Use Case 4: Advising Actions After Soil Health Analysis

This use case describes how the FasalGuard system provides actionable insights and recommendations to a Farmer after analyzing soil health parameters using machine learning models.

Table 2.4: Use Case 4: Advising Actions After Soil Health Analysis

Use Case Name	Advising Actions After Soil Health Analysis
Actors	Farmer (End User)
Description	After soil parameters such as pH, Nitrogen, Phosphorus, Potassium, and Moisture are collected and analyzed by the system, FasalGuard interprets the data using trained AI models to assess soil health status. Based on the analysis, it generates recommendations such as fertilizer type, quantity adjustments, irrigation schedules, and crop suitability. The insights are displayed to the Farmer through the dashboard and may also be delivered via SMS or email alerts.
Preconditions	1. Farmer must be logged into the system. 2. Soil data must be available for the farmer's region. 3. The user has already entered it's farm's location. 4. The soil analysis model must be trained and deployed successfully.
Postconditions	1. System successfully analyzes soil data and generates actionable insights. 2. Recommendations are displayed to the Farmer. 3. Alerts and reports are optionally sent via SMS or email.
Main Success Scenario	1. Farmer navigates to the Soil Health Analysis section on the dashboard. 2. System classifies the soil status (e.g., Healthy, Deficient, Acidic, Saline). 3. System generates context-aware recommendations for fertilizer type and amount. 4. Recommended irrigation schedule and soil treatment steps are displayed. 5. Farmer views the recommendations and plans the actions accordingly.
Alternate Flow	2. Poor model training might lead to inaccurate predictions. 5. If network connectivity is lost during recommendation generation, System saves the pending request and resumes processing once connection is restored.
Extensions	1. Model service temporarily unavailable. 2. Data format mismatch in uploaded soil data file.
Priority	High

2.1.5 Use Case 5: Weather Prediction

This use case allows a Farmer to view the weather forecast for their registered farm location. The system retrieves the region's coordinates from the user's stored farm data and uses an external weather API to predict and display weather conditions for the upcoming days.

Table 2.5: Use Case 5: Weather Prediction

Use Case Name	Weather Prediction
Actors	Farmer (End User)
Description	This use case enables a Farmer to access weather forecasts for their farm's region. The system retrieves the stored farm area details and fetches weather data from an integrated weather API. The prediction includes parameters such as temperature, humidity, precipitation, and wind speed for the next few days. The data is then visualized on the dashboard for easy interpretation.
Preconditions	1. Farmer must be logged into the system. 2. Farmer must have previously entered or registered their farm location/region. 3. The system must have a valid API connection to the weather data source.
Postconditions	1. Weather forecast data for the farm region is successfully retrieved and displayed. 2. The Farmer can view or refresh the forecast anytime from the dashboard.
Main Success Scenario	1. Farmer navigates to the <i>Dashboard</i> or <i>Weather</i> section. 2. System automatically retrieves the saved farm region. 3. System sends a request to the external weather API with location parameters. 4. API returns forecast data for the next few days. 5. System parses and formats the data into user-friendly graphs or tables. 6. Dashboard displays weather predictions including temperature, humidity, and rainfall. 7. Farmer views the forecast and plans agricultural activities accordingly.
Alternate Flow	1. Weather API service unavailable → System displays "Unable to fetch weather data. Please try again later." 2. Farmer has not entered farm location → System prompts the user to add farm details before proceeding. 3. Network connectivity issue → System retries or displays an offline mode notification.
Extensions	1. Invalid or outdated API key preventing data retrieval.
Priority	High

2.1.6 Use Case 6: Alerts/ Notifications

This use case describes how the FasalGuard system automatically generates and sends alerts to farmers regarding weather changes, crop stress, yield anomalies, or other critical agricultural conditions. The feature ensures timely delivery of essential information to help farmers take preventive or corrective actions.

Table 2.6: Use Case 6: Alerts and Notifications

Use Case Name	Alerts and Notifications
Actors	Farmers (Primary Actor)
Description	This use case defines how the FasalGuard system automatically monitors real-time data from weather APIs, soil inputs, and hybrid AI models to generate alerts for farmers. These alerts notify the user about upcoming weather events (e.g., heavy rainfall, heatwaves), crop stress conditions (e.g., drought, nutrient deficiency), or abnormal yield forecasts. Alerts are delivered via email, SMS, and displayed on the dashboard. The Admin can monitor, verify, or manually trigger alerts from the backend dashboard.
Preconditions	1. The user (Farmer) must have a registered and verified account. 2. The user's farm location and crop information must be stored in the system. 3. The system must have active access to weather and model prediction services.
Postconditions	1. The alert is successfully generated, delivered, and displayed on the farmer's dashboard.
Main Success Scenario	1. The system continuously monitors incoming weather, soil, and yield data. 2. AI-based thresholds and rule logic detect anomalies or risk conditions (e.g., drought, extreme temperature, pest indicators). 3. System generates an alert message containing type, severity, and recommended action. 4. The alert is displayed in the farmer's dashboard and sent via SMS or email. 5. The farmer reviews the alert and takes necessary preventive measures. 6. Admin can view all active and resolved alerts through the admin interface.
Alternate Flow	1. If a farmer has disabled SMS/email notifications → The alert is only shown on the dashboard. 2. If multiple alerts occur simultaneously → The system prioritizes based on severity and category. 3. If alert data cannot be fetched from API temporarily → The system retries or uses cached data for analysis.
Extensions	1. Notification service failure 2. Network issues delay alert delivery to users.
Priority	High
Special Requirements	1. Alerts must be sent instantly in case of anomaly detection.

2.1.7 Use Case 7: View Crop Field Forecasts / Crop Stress Heatmaps

Table 2.7: Use Case 7: View Crop Field Forecasts / Crop Stress Heatmaps

Use Case Name	View Crop Field Forecasts / Crop Stress Heatmaps
Scope	FasalGuard
Level	User Goal Level
Actors	Farmer (Primary Actor)
Stakeholders and Interests	Farmers: Want accurate, visual insights into crop health and early warnings for stress or low yield areas. Administrators: Require performance monitoring of AI models and management of satellite imagery datasets. Government Agencies: Interested in regional stress patterns to plan interventions and optimize agricultural policies.
Preconditions	1. The Farmer must be logged into the system. 2. Farm location must be registered in the system. 3. The system must have access to the satellite imagery API (e.g., Sentinel-2 or Google Earth Engine). 4. The AI model for stress detection must be trained and deployed.
Success Guarantee (Postconditions)	1. The system successfully retrieves and analyzes recent satellite imagery for that region's field. 2. Stress heatmaps and forecast results are displayed on the user dashboard. 3. The analysis results are stored in the database for historical comparison and trend visualization.
Main Success Scenario	1. Farmer logs into the system and navigates to the "Crop Forecasts / Heatmaps" section. 2. System retrieves the user's registered farm boundaries. 3. System fetches the most recent satellite images for the specified area. 4. The backend sends the imagery data to the trained CNN-based crop stress detection model. 5. Model identifies crop stress zones. 6. System generates visual heatmaps representing crop health status (Healthy, Moderate Stress, Severe Stress). 7. Dashboard displays the generated heatmap and predicted yield forecast for the selected field. 8. Farmer reviews insights and plans accordingly.
Extensions	NULL
Special Requirements	1. The system should render high-resolution heatmaps efficiently. 2. Display should be responsive across web interfaces. 3. Secure access to remote satellite API keys and imagery.
Frequency of Occurrence	Frequent: Expected to be used weekly or after major environmental events (rainfall, droughts, pest outbreaks).

2.1.8 Use Case 8: Predict Disaster and Its Precautionary Measures

Table 2.8: Use Case 8: Predict Disaster and Its Precautionary Measures

Use Case Name	Predict Disaster and Its Precautionary Measures
Scope	FasalGuard
Level	User Goal Level
Actors	Farmer (Primary Actor)
Stakeholders and Interests	Farmers: Want timely and reliable predictions of climate-related disasters (e.g., drought, flood, heatwave) to protect crops and plan preventive actions. System Administrator: Ensures model reliability, data pipeline integrity, and continuous updates of meteorological datasets.
Preconditions	1. The Farmer must be logged into the system. 2. The Farmer's region and crop type must already be registered. 3. The system must have access to real-time and historical climate datasets (temperature, rainfall, humidity, wind speed, etc.). 4. The climate prediction model must be deployed and trained with validated meteorological data.
Success Guarantee (Postconditions)	1. The system successfully analyzes climatic parameters and predicts potential risks or disasters. 2. The system displays preventive measures and actionable insights to minimize damage. 3. Alerts are shared in case of disaster prediction.
Main Success Scenario	1. Farmer logs into the system and navigates to the "Disaster Prediction" section on the dashboard. 2. System retrieves the Farmer's registered location and crop type. 3. System fetches real-time weather data (temperature, rainfall, humidity, and wind patterns) via integrated API. 4. The backend passes this data to the trained Climate Risk Prediction Model. 5. The model evaluates parameters and identifies potential climate threats (e.g., drought, flood, heatwave). 6. System generates and displays disaster likelihood percentages with graphical indicators. 7. The system suggests precautionary measures such as irrigation adjustments, fertilizer scheduling, crop cover, or early harvesting. 8. Farmer reviews recommendations and can choose to receive notifications via email/SMS.
Extensions	1. If the Farmer's location is not registered then the System prompts user to add or confirm region details.
Special Requirements	1. All recommendations should be localized according to soil and crop type. 2. Disaster alerts should trigger automated notifications for the Farmer.
Technology and Data Variations List	NULL 22
Frequency of Occurrence	Frequent: Used daily or weekly depending on environmental volatility and farming season. Farmers are expected to consult the predictions before major agricultural activities.

2.2 Functional Requirements

This section describes the functional requirements of the FasalGuard. These requirements define the specific functionalities that the system must perform to fulfill the goals of precision agriculture through data integration, AI modeling, and decision support. The requirements are organized by modules to align with the system's development structure and iteration plan.

2.2.1 Module 1: Data Collection

Following are the functional requirements for Module 1:

1. **FR1:** The system shall collect at least ten years of historical climate data for Punjab using APIs such as NASA POWER and production data from Punjab Bureau of Statistics (PBC).
2. **FR2:** The system shall acquire annual crop yield statistics for major crops including wheat, rice, maize, cotton, and sugarcane.
3. **FR3:** The system shall gather soil health parameters such as pH, nitrogen content, moisture, and organic matter from pilot fields.
4. **FR4:** The system shall retrieve Sentinel-2 satellite imagery for field-level vegetation analysis and crop classification.
5. **FR5:** The collected datasets shall be stored in a centralized and structured database repository for further preprocessing and analysis.

2.2.2 Module 2: Data Cleaning and Preprocessing

Following are the functional requirements for Module 2:

1. **FR1:** The system shall identify and handle missing or inconsistent data across climate, soil, and yield datasets.
2. **FR2:** The system shall normalize and scale numeric features (e.g., temperature, rainfall, soil metrics) for cross-regional comparability.
3. **FR3:** The system shall preprocess Sentinel-2 imagery by applying cloud masking, band selection, and spatial resampling techniques.
4. **FR4:** The system shall output cleaned and structured datasets ready for model training and validation.

2.2.3 Module 3: Yield Forecasting (Iteration 1)

Following are the functional requirements for Module 3:

1. **FR1:** The system shall train machine learning and deep learning models (e.g., LSTM) using preprocessed time-series datasets for yield prediction.
2. **FR2:** The system shall evaluate trained models using performance metrics such as RMSE, MAE, and R^2 .
3. **FR3:** The system shall generate yield forecasts based on historical climate and production data.
4. **FR4:** The system shall visualize yield forecasts and trends on the web dashboard for user interpretation.

2.2.4 Module 4: Soil Integration and Satellite Images Collection (Iteration 2)

Following are the functional requirements for Module 4:

1. **FR1:** The system shall train predictive models using soil health parameters to assess fertility and productivity across regions.
2. **FR2:** The system shall evaluate soil model accuracy using standard regression metrics (RMSE, MAE, R^2).
3. **FR3:** The system shall combine soil-based predictions with climate and yield forecasts for enhanced accuracy.
4. **FR4:** The system shall continuously collect real-time satellite imagery for future CNN model training.

2.2.5 Module 5: Satellite Images and Hybrid Model Development (Iteration 3)

Following are the functional requirements for Module 5:

1. **FR1:** The system shall implement CNN-based analysis of Sentinel-2 imagery to detect vegetation health and crop stress patterns.
2. **FR2:** The system shall integrate outputs from LSTM (climate), Random Forest (soil), and CNN (satellite imagery) models into a unified hybrid architecture.

3. **FR3:** The system shall provide near real-time yield and stress predictions using the hybrid model.
4. **FR4:** The system shall trigger automated alerts for early warnings related to drought, nutrient deficiency, or pest risks.
5. **FR5:** The system shall generate interactive maps and risk visualizations on the dashboard.
6. **FR6:** The system shall optimize hybrid model inference time for scalability and responsiveness.

2.2.6 Module 6: Stress Detection and Climate-Risk Forecasting

Following are the functional requirements for Module 6:

1. **FR1:** The system shall integrate real-time weather data with historical trends to detect anomalies.
2. **FR2:** The system shall identify early signs of drought, floods, or pest outbreaks using predictive analysis.
3. **FR3:** The system shall produce localized stress heatmaps and risk visualizations for affected regions.
4. **FR4:** The system shall generate data-driven recommendations for irrigation scheduling and fertilizer optimization.
5. **FR5:** The system shall support temporal forecasting to anticipate near-future climate impacts on crops.

2.2.7 Module 7: Alerts, Dashboard, and Deployment (Iteration 4)

Following are the functional requirements for Module 7:

1. **FR1:** The system shall provide a responsive, user-friendly dashboard for Farmers and Admins to visualize agricultural insights.
2. **FR2:** The system shall enable farmers to receive SMS or email alerts related to weather risks, yield drops, or crop stress conditions.
3. **FR3:** The system shall support admin-level access for managing user data, farm records, and system logs.

4. **FR4:** The system shall allow seamless integration of new models and datasets for scalability.
5. **FR5:** The system shall be deployed on a cloud or web server environment for regional access and pilot testing.

2.3 Non-Functional Requirements

This section outlines the non-functional requirements (NFRs) that define the system's overall quality attributes, including performance, reliability, usability, and security. These requirements ensure that FasalGuard operates efficiently, securely, and consistently under real-world conditions.

2.3.1 Security

The system must ensure the confidentiality and integrity of user data through secure authentication and encrypted communication. Passwords and sensitive user data will be encrypted using industry-standard algorithms. Role-based access control (RBAC) will restrict admin-level operations and prevent unauthorized access to backend components and the database.

2.3.2 Usability

FasalGuard's web interface will be designed to be simple, intuitive, and accessible for users with minimal technical background. Farmers should be able to navigate features such as weather prediction, soil health reports, and alerts without training. The interface will follow consistent color schemes and icons for better readability and user experience.

2.3.3 Responsiveness

The application must adapt seamlessly across devices, including desktops, tablets, and mobile phones. All UI elements, dashboards, and charts should render dynamically, ensuring a smooth experience on varying screen resolutions and browsers.

2.3.4 Availability

The system shall maintain a minimum of 99% uptime. All critical backend services, including the model inference engine and alert system, should be designed for fault toler-

ance. In case of downtime or network failure, cached data and recent predictions should remain accessible to users.

2.3.5 Reliability

The platform must consistently deliver accurate results and maintain stability under varying workloads. Data synchronization between backend and frontend components must be robust to prevent corruption or loss during API calls or model execution.

2.3.6 Performance

All key operations, including data retrieval, weather prediction, and model inference, should execute within 3–5 seconds on a standard network connection (20 Mbps or above). The backend should handle at least 100 concurrent requests without significant degradation in response time.

2.3.7 Model Accuracy

Each predictive model (Climate, Soil Health, Satellite Imagery) must achieve at least 85–90% accuracy in validation tests. Models will be periodically retrained with new datasets to maintain precision and adaptability to evolving climate and crop conditions.

2.3.8 Maintainability

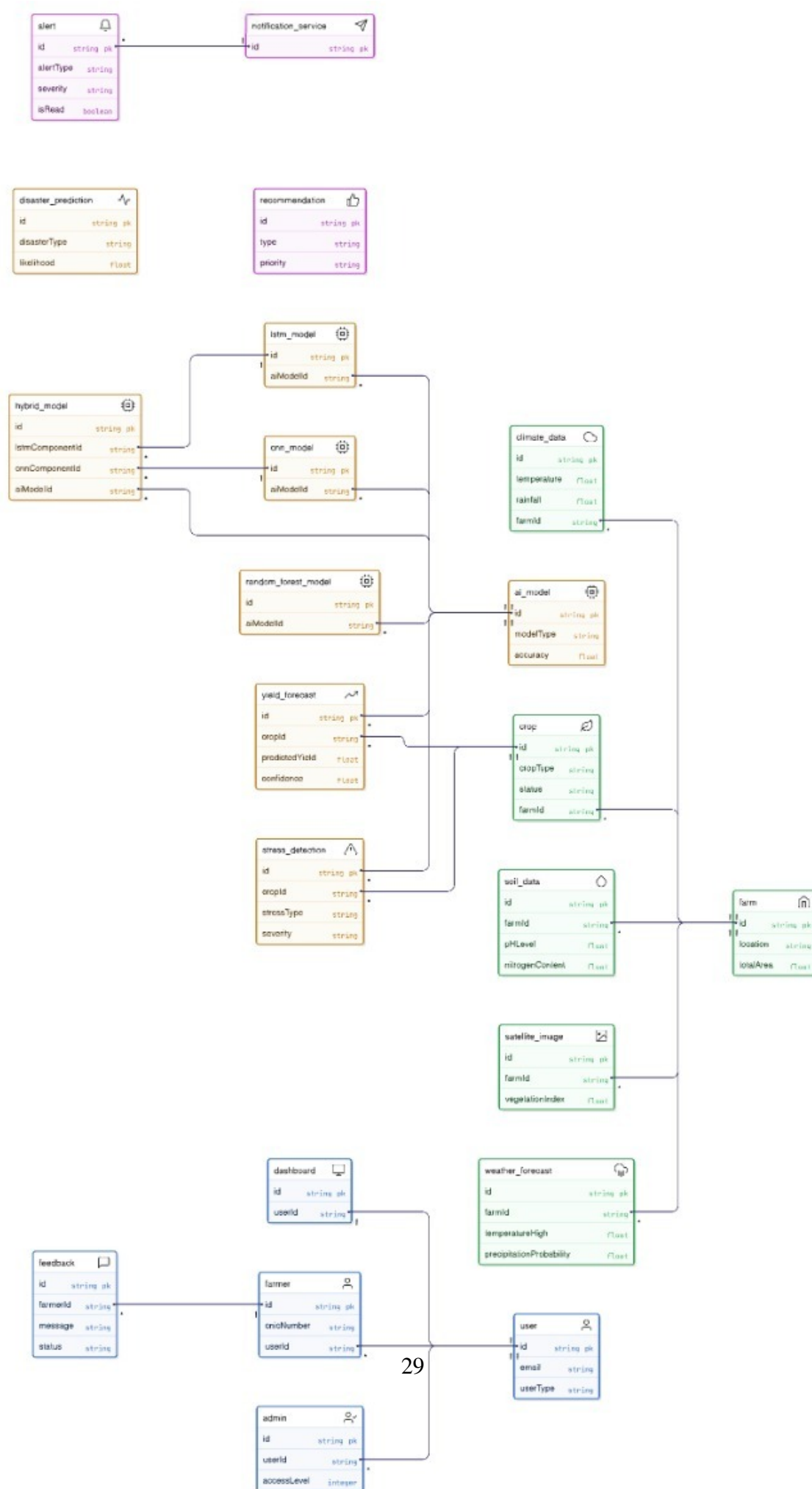
The system architecture should follow modular design principles, enabling easy updates to models, APIs, and UI components. Version control will be managed via GitHub, and all code will follow standardized documentation practices.

2.3.9 Scalability

FasalGuard should support horizontal scaling to handle increased data and users. The system must be capable of integrating additional regions, crops, and datasets without major reconfiguration or downtime.

2.4 Domain Model

The Domain Model provides a conceptual representation of the key entities within the FasalGuard system and their relationships. It captures the real-world objects and concepts relevant to precision agriculture, including users, farms, crops, environmental data, and AI-driven predictions. This model serves as the foundation for database design and system architecture.



Key Domain Entities:

- **User:** Represents system users including farmers and administrators with authentication details and role-based access.
- **Farm:** Contains geographical location, crop information, and links to associated data sources.
- **Crop:** Defines crop types, planting dates, and growth status for yield tracking.
- **SoilData:** Stores soil health parameters such as pH, nitrogen content, and moisture levels.
- **ClimateData:** Captures weather-related information including temperature, rainfall, and humidity.
- **SatelliteImage:** Manages remotely sensed data from Sentinel-2 for vegetation monitoring.
- **YieldForecast:** AI-generated predictions for expected crop production.
- **StressDetection:** Identifies crop stress conditions such as drought or nutrient deficiency.
- **Alert:** Notification system for critical events and recommendations.
- **AIModel:** Abstract representation of machine learning models (LSTM, CNN, Random Forest) used for predictions.

The domain model illustrates inheritance relationships (e.g., Farmer and Admin extending User), composition (Farm containing Crops), and associations between data entities and predictive models.

Chapter 3

System Overview

Give a general description of the functionality, context, and design of your project. FasalGuard is a comprehensive AI-driven precision agriculture platform designed to empower farmers in Pakistan's Punjab region with data-integrated insights for crop management, yield optimization, and risk mitigation. By leveraging heterogeneous data sources such as satellite imagery, climate records, and soil metrics, the system employs hybrid machine learning models to deliver real-time forecasts, stress detections, and actionable recommendations via a user-friendly web dashboard and SMS alerts.

Provide any background information if necessary. Traditional farming practices in Pakistan face challenges from climate variability, resource scarcity, and limited technological access. FasalGuard addresses these by bridging advanced geospatial and AI technologies with on-ground needs, drawing from global datasets like NASA POWER and Sentinel-2 to enable sustainable agriculture.

3.1 Architectural Design

Given below is the architectural diagram for FasalGuard. 3-tier architecture is being used.

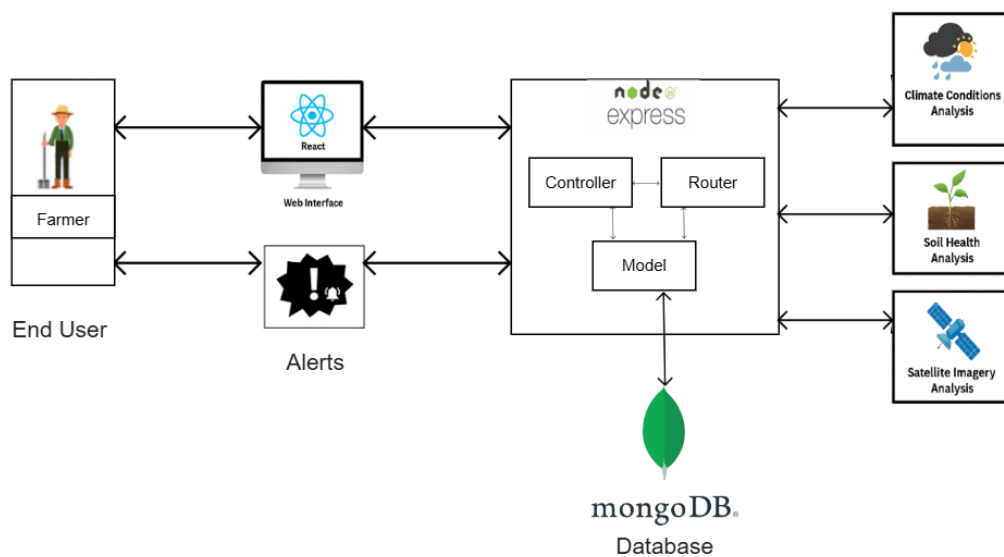


Figure 3.1: Architectural Diagram

3.2 Design Models

The design of FasalGuard follows the **Object-Oriented Development Approach (OODA)** due to its modular nature, clear separation of concerns, and need for scalability. Since the system consists of multiple interacting modules like data processing, AI models, and a web interface so an object-oriented structure allows each component (class or module) to encapsulate data and behavior independently.

This approach enhances maintainability, reusability, and extensibility. For instance, the backend classes such as `UserController`, `ModelHandler`, and `AlertManager` can easily be extended or replaced without disrupting the rest of the system. Moreover, object-oriented design aligns naturally with frameworks like `Node.js` (Express) and `React.js`, which utilize modular components and encapsulated logic, as well as with the model-based architecture used in AI/ML pipelines.

The following subsections present four major design models created under the OODA: the Domain Model, Activity Diagram, Class Diagram, and Class-Level Sequence Diagram.

3.2.1 Activity Diagram

The Activity Diagram illustrates the overall system workflow of FasalGuard, showing how users (farmers or administrators) interact with the system, from logging in to receiving data-driven insights. The flow highlights processes such as data input, weather and soil analysis, model prediction, and alert generation.

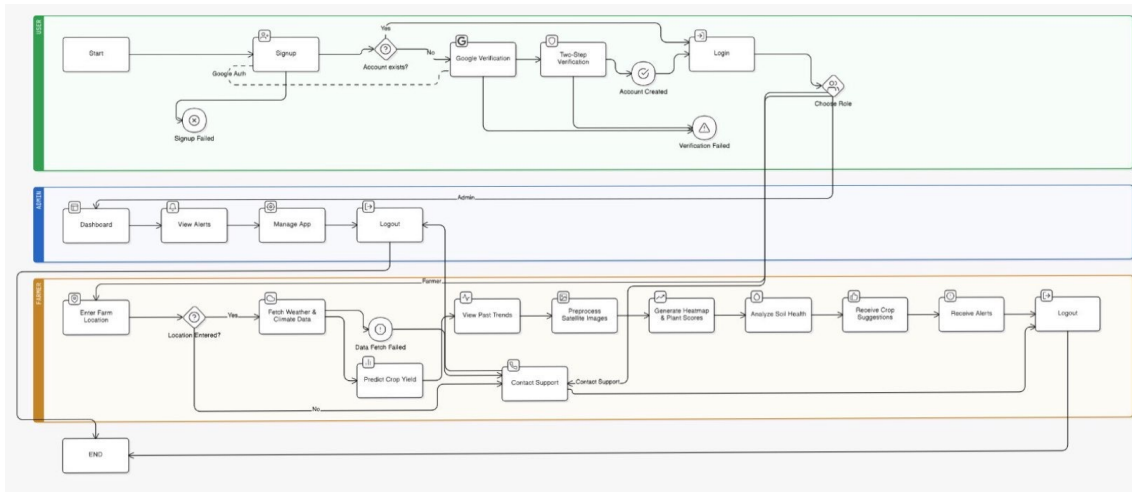


Figure 3.2: Activity Diagram of FasalGuard

Description: The user initiates interaction by logging in or signing up. After authentication, the system retrieves the user's farm data and processes it through different modules: soil health analysis, climate-based prediction, and satellite imagery interpretation. The resulting insights are integrated into a unified dashboard and trigger alerts if abnormal conditions (e.g., drought or nutrient deficiency) are detected.

3.2.2 Class Diagram

The Class Diagram represents the static structure of the system, detailing the major classes, their attributes, operations, and interrelationships. It reflects how the backend components, frontend elements, and models interact in an object-oriented context.

3. System Overview

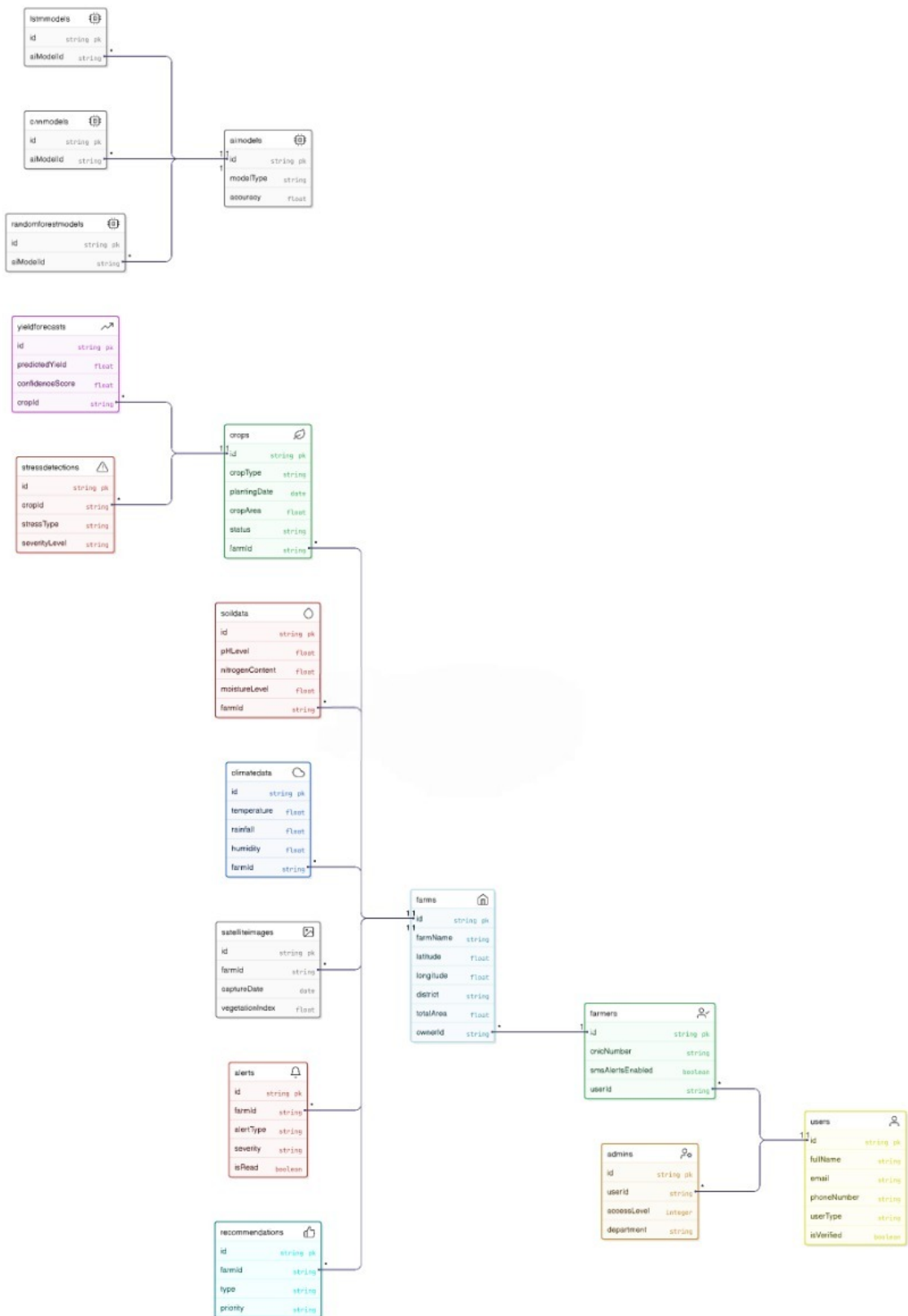


Figure 3.3: Class Diagram of FasalGuard

Key Classes:

- **User:** Stores user data (name, email, password, role) and manages authentication.
- **Farm:** Maintains region, soil parameters, and crop type information.
- **WeatherAPI:** Handles weather data retrieval and integration with external APIs.
- **SoilHealthModel:** Analyzes soil nutrient data and returns health status.
- **ClimateModel:** Predicts disaster risks such as drought or flood.
- **SatelliteModel:** Processes Sentinel-2 imagery and identifies crop stress.
- **AlertManager:** Generates and delivers warnings or recommendations.
- **DashboardController:** Handles user dashboard data visualization and analytics.

The diagram demonstrates class interactions through associations and dependencies, ensuring separation between data entities and functional logic.

3.2.3 Class-Level Sequence Diagram

Description: Class-level sequence diagram demonstrates the workflow of a specific use case.

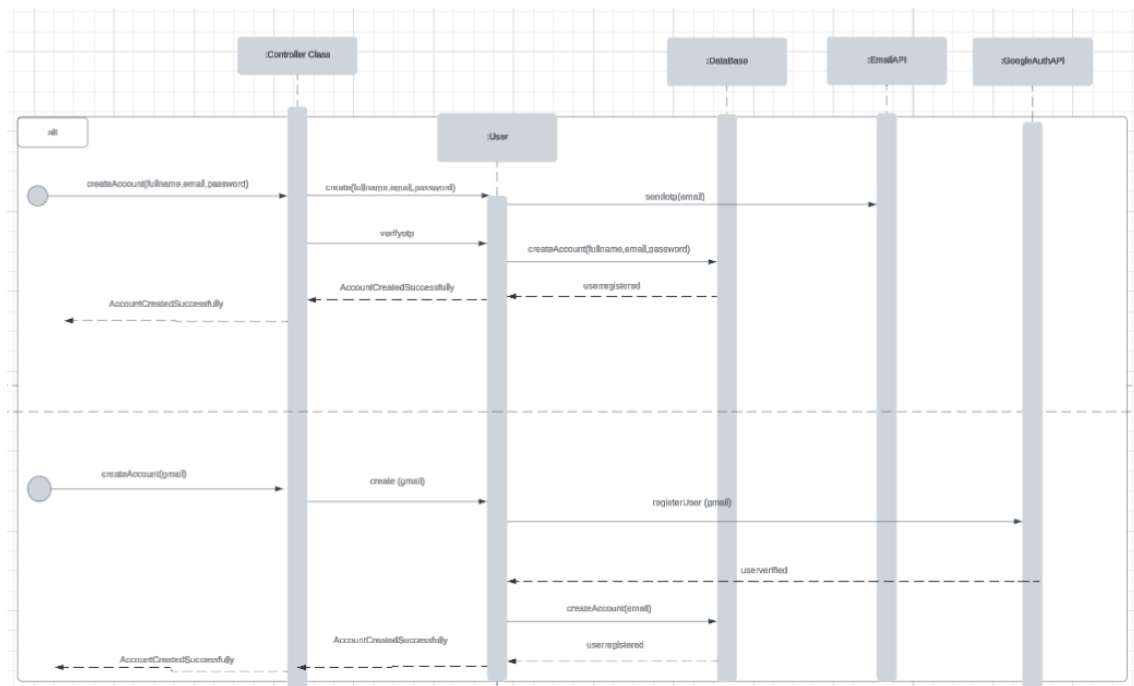


Figure 3.4: Class-Level Sequence Diagram of SignUp/CreateanAccount

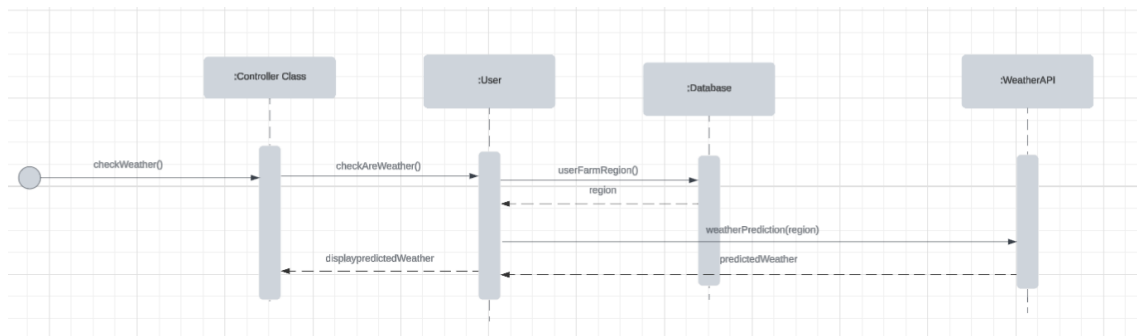


Figure 3.5: Class-Level Sequence Diagram of Weather Prediction

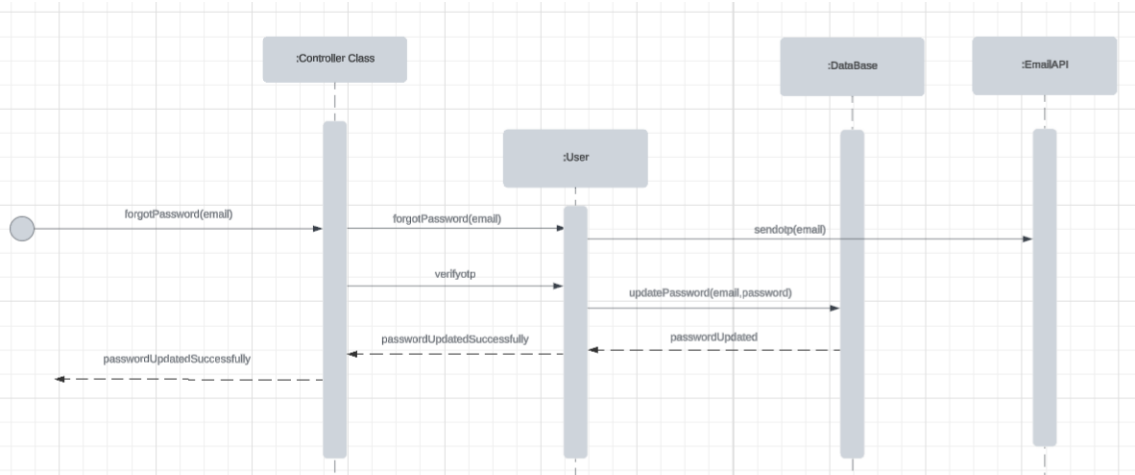


Figure 3.6: Class-Level Sequence Diagram of Forgot Password

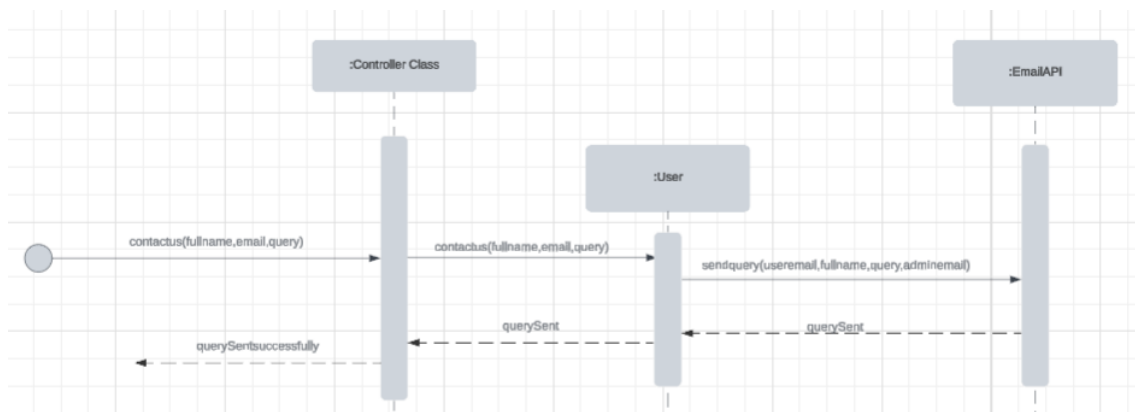


Figure 3.7: Class-Level Sequence Diagram of Feedback/Ask Query

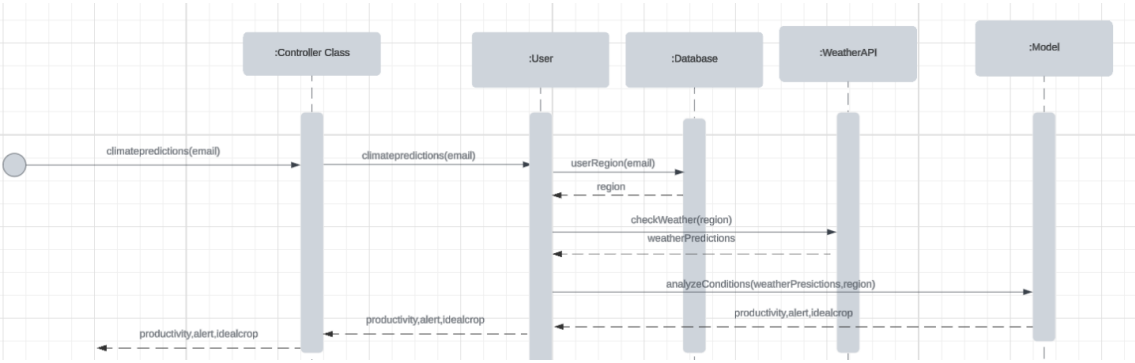


Figure 3.8: Class-Level Sequence Diagram of LSTM model

3.3 Data Design

The data design of FasalGuard defines how agricultural information is captured, stored, processed, and organized to support AI-driven predictions and real-time decision-making. The system integrates heterogeneous data sources including climate records, soil metrics, satellite imagery, and crop production statistics. These diverse datasets are transformed into structured formats suitable for machine learning models and user-facing applications.

3.3.1 Database Architecture

FasalGuard employs a **relational database management system (RDBMS)** using **PostgreSQL** to ensure data consistency, integrity, and support for complex queries. The database is organized into multiple interconnected tables that represent users, farms, crops, environmental data, and AI-generated predictions.

Primary Database Tables:

- **users:** Stores user authentication credentials, contact information, and role-based access control (farmer, admin, agricultural officer).
- **farms:** Contains farm-specific details including geographical coordinates (latitude, longitude), district, region, and total cultivated area.
- **crops:** Tracks crop types (wheat, rice, maize, cotton, sugarcane), planting dates, expected harvest dates, and current status.
- **soil_data:** Records soil health parameters such as pH level, nitrogen, phosphorus, potassium content, moisture levels, and organic matter.
- **climate_data:** Captures historical and real-time weather information including temperature, humidity, rainfall, and wind speed sourced from NASA POWER API.
- **satellite_images:** Stores metadata and URLs for Sentinel-2 imagery along with calculated vegetation indices (NDVI) and capture timestamps.
- **weather_forecasts:** Maintains short-term weather predictions with daily high/low temperatures and precipitation probabilities.
- **yield_forecasts:** Stores AI-generated yield predictions with confidence scores and model version identifiers.
- **stress_detection:** Records detected crop stress conditions including stress type (drought, nutrient deficiency, pest), severity level, and affected area percentage.

- **disaster_predictions:** Contains climate-risk forecasts for droughts, floods, heat-waves, and storms with likelihood percentages.
- **alerts:** Manages notification records with alert types, severity levels, delivery methods, and read/resolved status.
- **recommendations:** Stores actionable insights for irrigation, fertilization, pest control, and harvest timing with priority classifications.
- **feedback:** Captures user queries, feedback submissions, and admin responses for continuous system improvement.
- **model_performance:** Tracks AI model evaluation metrics (RMSE, MAE, R²) and training metadata for performance monitoring.

3.3.2 Entity-Relationship Model

The Entity-Relationship (ER) diagram illustrates the logical structure of the database and relationships between entities. Key relationships include:

- **One-to-Many:** A single user can own multiple farms; a farm can contain multiple crops.
- **One-to-Many:** Each farm is associated with multiple soil data records, climate data entries, and satellite images for temporal analysis.
- **One-to-Many:** Each crop links to multiple yield forecasts and stress detection records over time.
- **Many-to-Many:** Alerts can be triggered by multiple farms and delivered to multiple users based on subscription preferences.

3.3.3 Data Processing Pipeline

The system implements a multi-stage data processing pipeline to transform raw inputs into actionable insights:

Stage 1: Data Collection

- Historical climate data retrieved from NASA POWER API spanning 10+ years.

- Annual crop production statistics collected from Pakistan Bureau of Statistics.
- Soil samples analyzed from pilot fields across Punjab districts.
- Sentinel-2 satellite imagery acquired via Google Earth Engine API.

Stage 2: Data Cleaning and Preprocessing

- Missing value imputation using forward-fill and interpolation techniques.
- Outlier detection and removal using statistical methods (IQR, Z-score).
- Feature normalization and scaling for cross-regional comparability.
- Satellite image preprocessing including cloud masking, band selection, and resampling.

Stage 3: Feature Engineering

- Calculation of vegetation indices (NDVI, EVI) from satellite bands.
- Temporal aggregation of climate variables (7-day, 14-day, 30-day moving averages).
- Soil fertility index computation based on NPK ratios and pH levels.
- Derivation of growing degree days (GDD) for crop phenology modeling.

Stage 4: Model Training and Inference

- LSTM networks process time-series climate data for yield forecasting.
- Random Forest models analyze soil parameters for fertility assessment.
- CNN architectures extract spatial features from satellite imagery for stress detection.
- Hybrid model integration combines outputs for comprehensive predictions.

3.3.4 Data Storage and Access

FasalGuard employs a hybrid storage strategy to optimize performance and scalability:

Structured Data Storage:

- Relational database (PostgreSQL) for transactional data, user information, and prediction results.
- Indexed queries for fast retrieval based on farm location, crop type, and date ranges.
- Foreign key constraints ensure referential integrity across related tables.

Unstructured Data Storage:

- Cloud object storage (AWS S3 or Google Cloud Storage) for satellite imagery and large raster files.
- File metadata stored in the database with references to cloud storage URLs.
- Content Delivery Network (CDN) integration for efficient image delivery to the dashboard.

Caching Layer:

- Redis cache for frequently accessed data such as weather forecasts and dashboard analytics.
- Session management and temporary storage for real-time prediction requests.
- Cache invalidation strategies based on data freshness requirements.

3.3.5 Data Security and Privacy

Given the sensitive nature of agricultural data, FasalGuard implements robust security measures:

- **Encryption:** All user passwords hashed using bcrypt; sensitive data encrypted at rest and in transit (SSL/TLS).
- **Access Control:** Role-based permissions restrict data access based on user type (farmer, admin, researcher).

- **Data Anonymization:** Aggregated regional statistics provided to policymakers without exposing individual farm data.
- **Backup and Recovery:** Automated daily database backups with point-in-time recovery capabilities.
- **Audit Logging:** System tracks data access patterns and modification history for security monitoring.

3.3.6 Data Integration and APIs

The system integrates multiple external data sources through RESTful APIs:

- **NASA POWER API:** Retrieves historical and near-real-time climate data for any geographical coordinate.
- **Google Earth Engine:** Provides programmatic access to Sentinel-2 satellite imagery and cloud-based geospatial processing.
- **OpenWeather API:** Supplies short-term weather forecasts for alert generation and irrigation recommendations.
- **ISRIC SoilGrids:** Offers global soil property data for regions where field samples are unavailable.

Internal APIs expose prediction endpoints for the frontend dashboard:

- **POST /api/predict/yield:** Accepts farm and crop parameters, returns yield forecast.
- **GET /api/stress/heatmap:** Generates crop stress visualization for a specified farm boundary.
- **GET /api/alerts/user/:userId:** Retrieves active alerts for a specific user.
- **POST /api/recommendations:** Generates data-driven farming recommendations based on current conditions.

3.3.7 Scalability Considerations

To accommodate future growth and increased data volume, the data design incorporates:

- **Horizontal Scaling:** Database sharding based on geographical regions (districts) to distribute load.
- **Time-Series Partitioning:** Climate and satellite data partitioned by year/month for efficient archival and query performance.
- **Asynchronous Processing:** Background job queues (Celery, Bull) for computationally intensive model inference tasks.
- **Microservices Architecture:** Separation of data ingestion, processing, and serving layers for independent scaling.